

Gang Scheduling in Kubernetes

What's Happening and What Comes Next



Content

1.35

- ✓ Gang Scheduling Support in Kubernetes:
Introduce **Workload** (Alpha 1)
- ✓ Opportunistic batching (Beta)

1.37+

- PDB for Multi-Pod Replicas
- Integrate Workload APIs with
JobSet and LWS

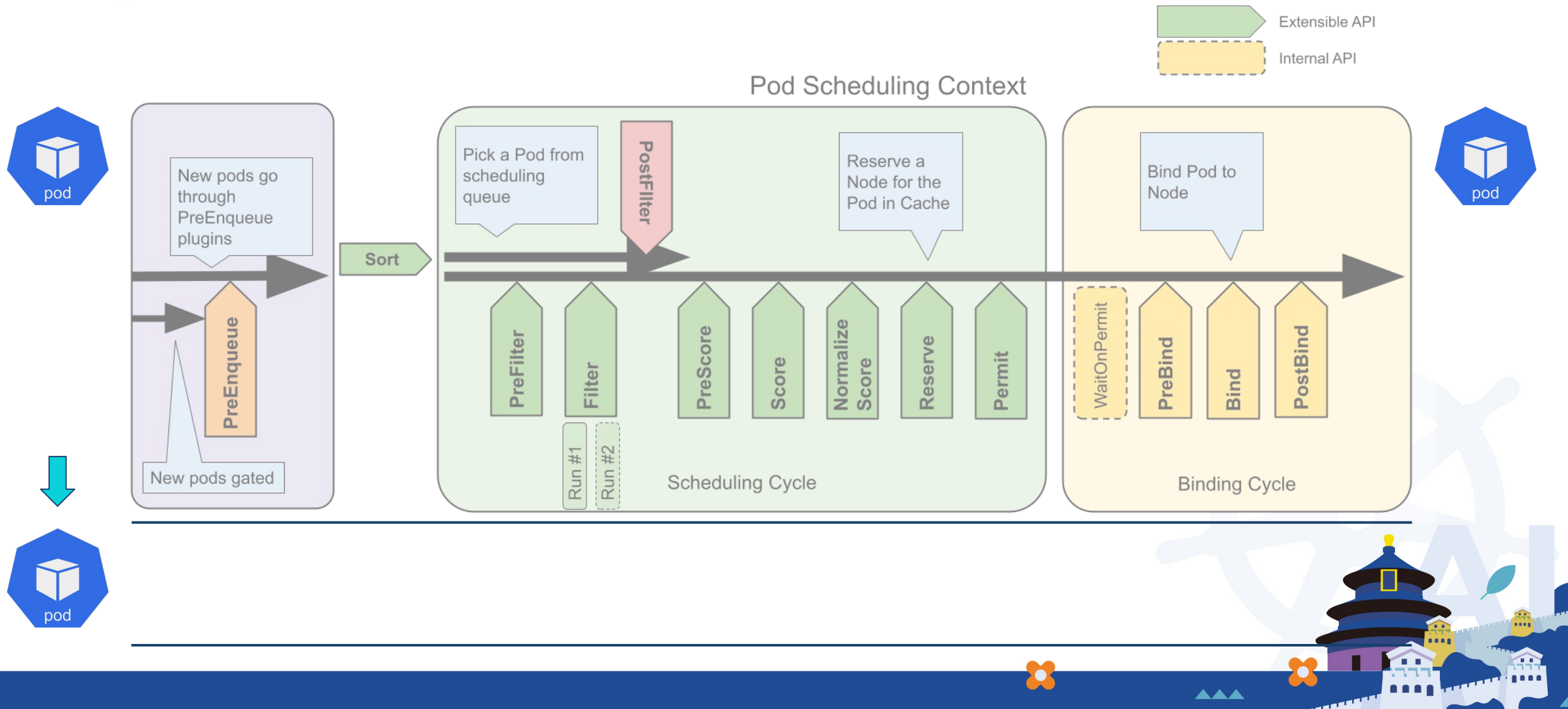
1.36

- Decouple **PodGroup** from Workload API
- Integrate Workload APIs with Job Controller
- DRA: ResourceClaim Support for Workloads
- Workload-aware preemption in scheduler
- Topology-aware workload scheduling

1.35



Pod Scheduling



KEP-4671 (before 1.36): Pod can specify its group. New core type representing a workload to kube-scheduler. Gang scheduling by kube-scheduler for each pod group within workload objects.

Static Template ?

```
kind: Workload
metadata:
  name: wd
spec:
  podGroups:
  - name: workers
    minCount: 8
```

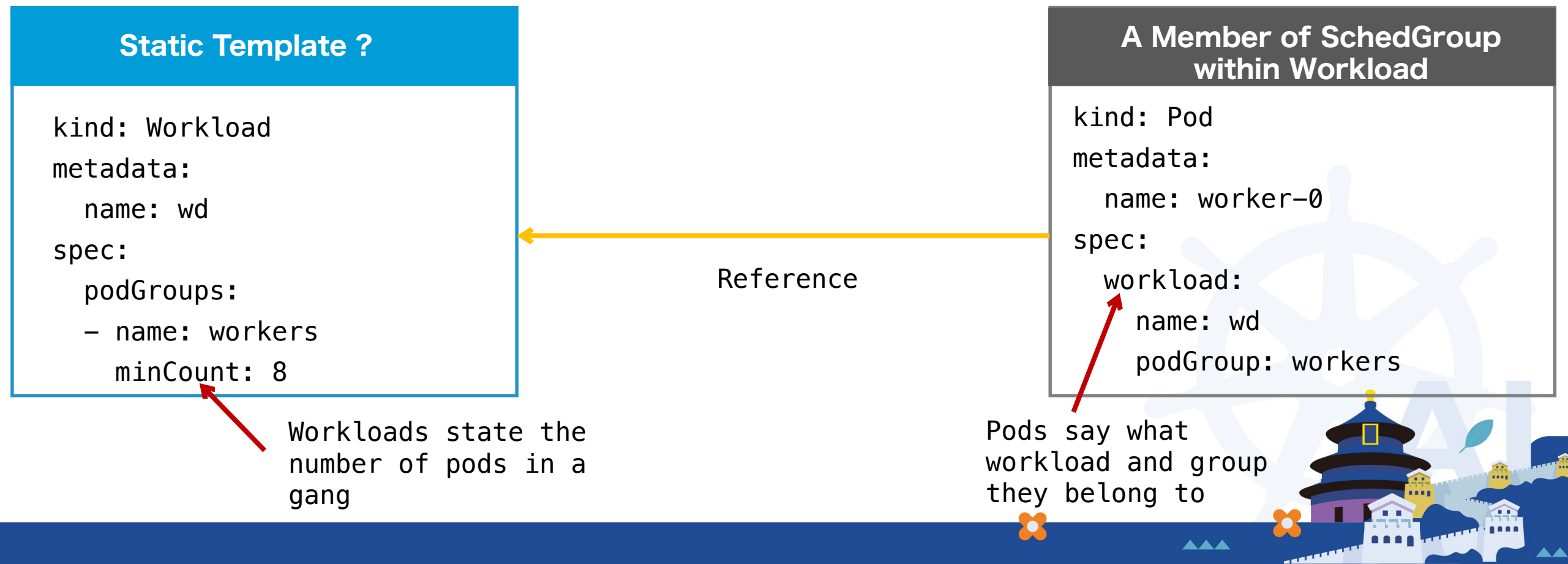
Workloads state the number of pods in a gang

A Member of SchedGroup within Workload

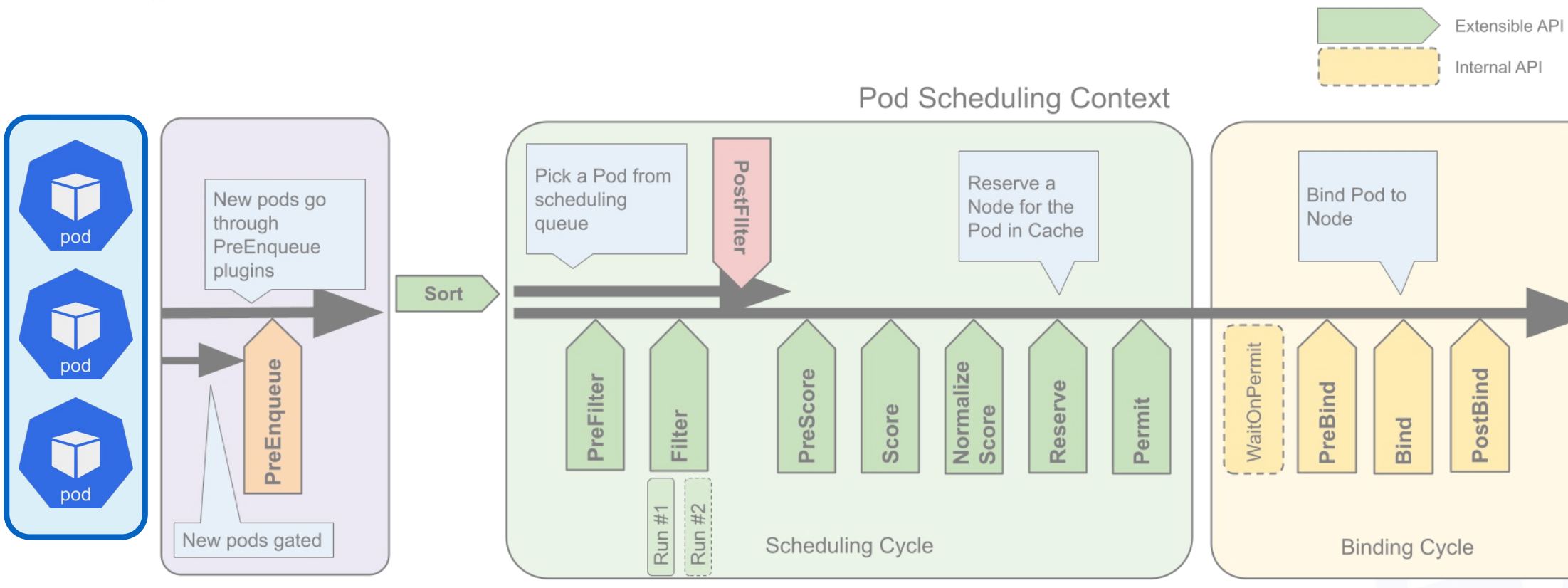
```
kind: Pod
metadata:
  name: worker-0
spec:
  workload:
    name: wd
    podGroup: workers
```

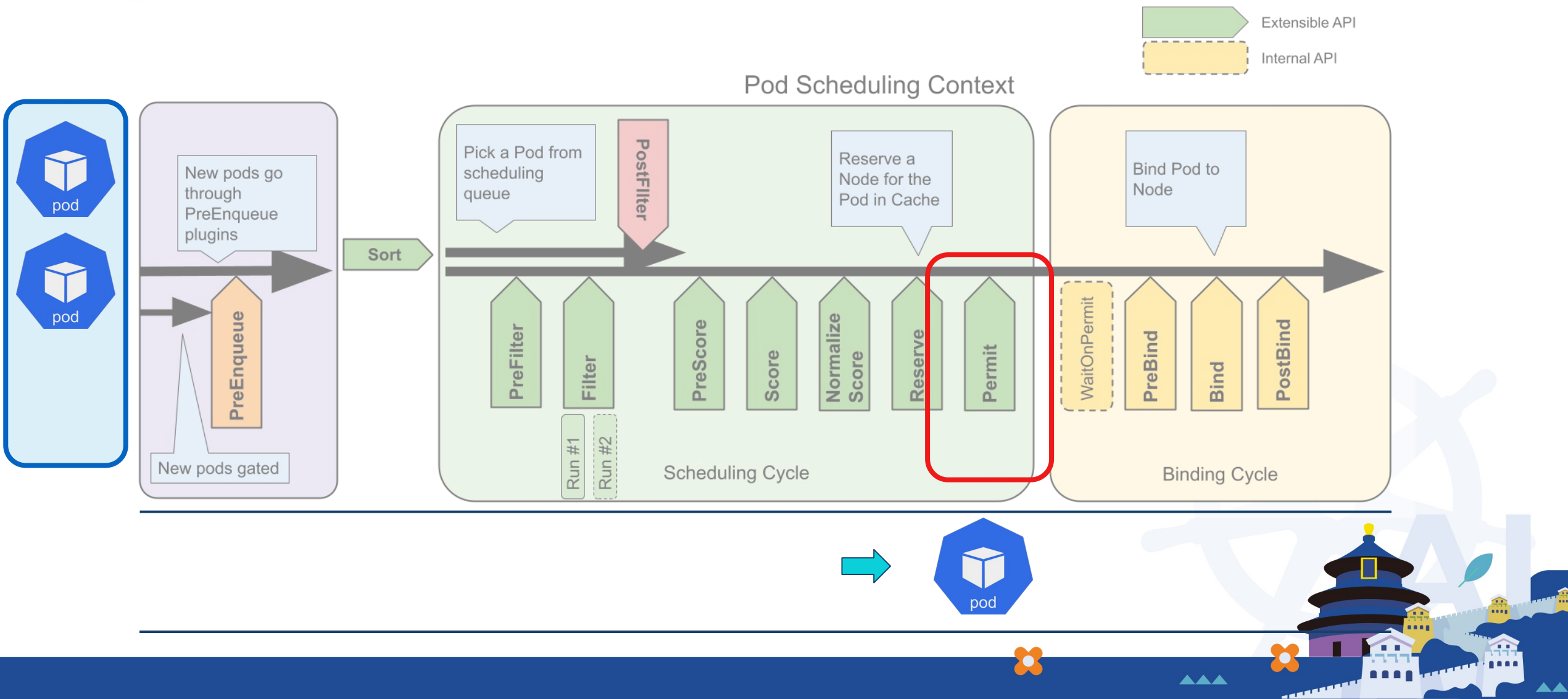
Pods say what workload and group they belong to

Reference

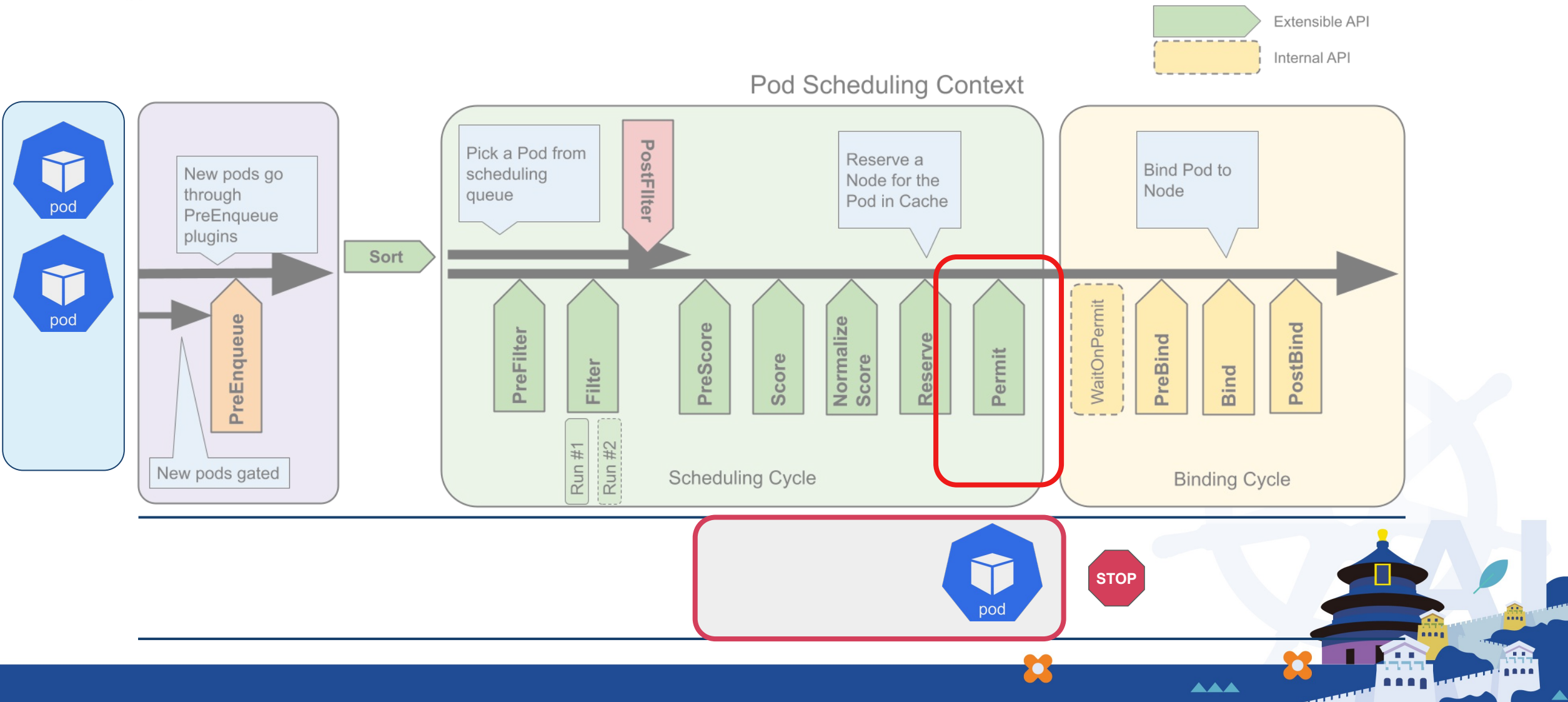


10-4

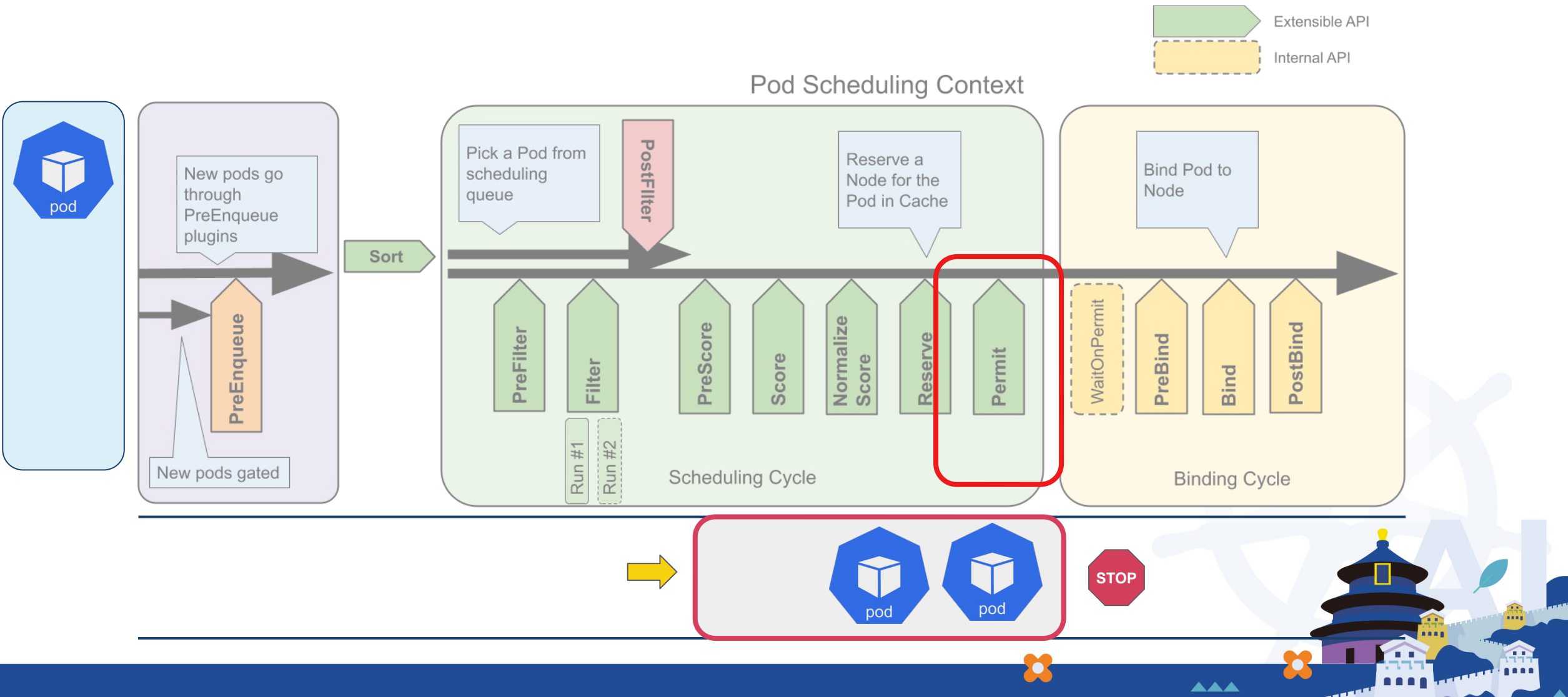




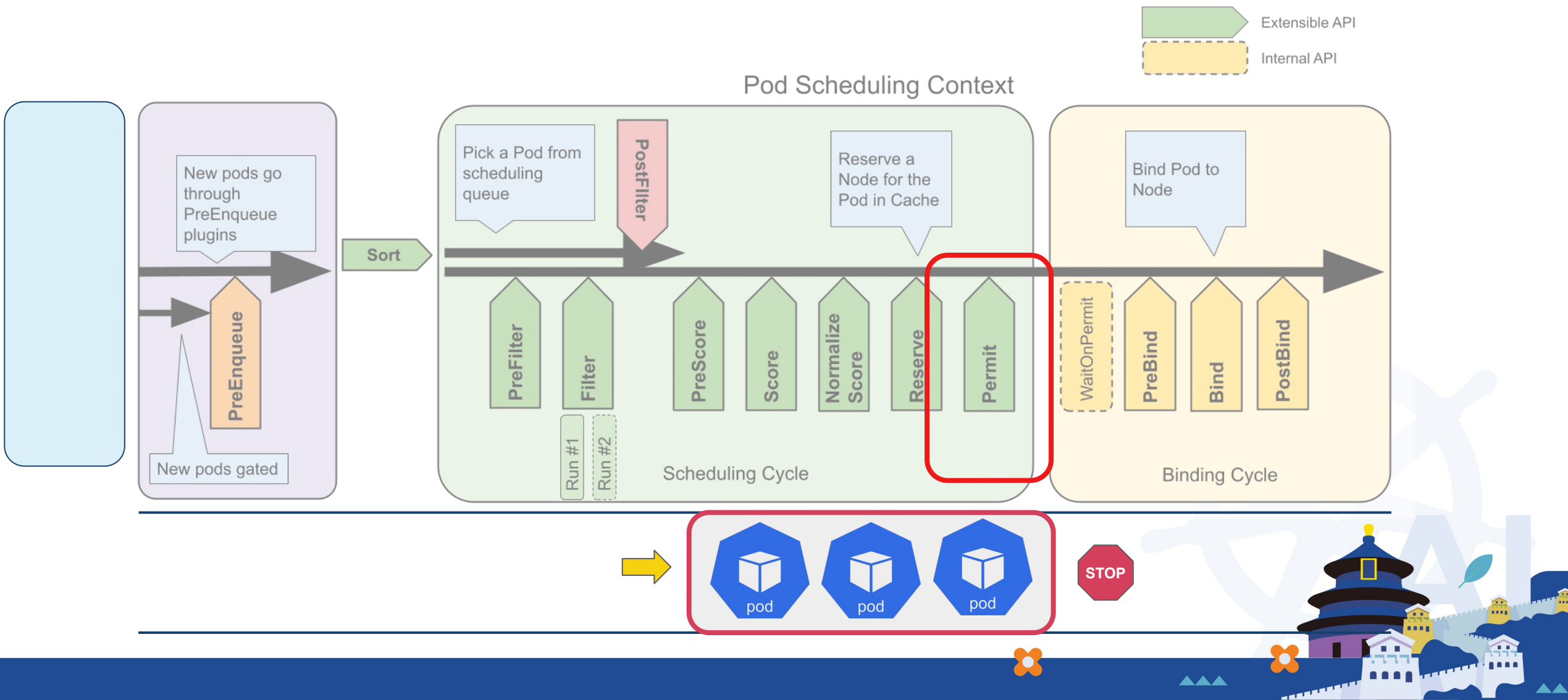
100



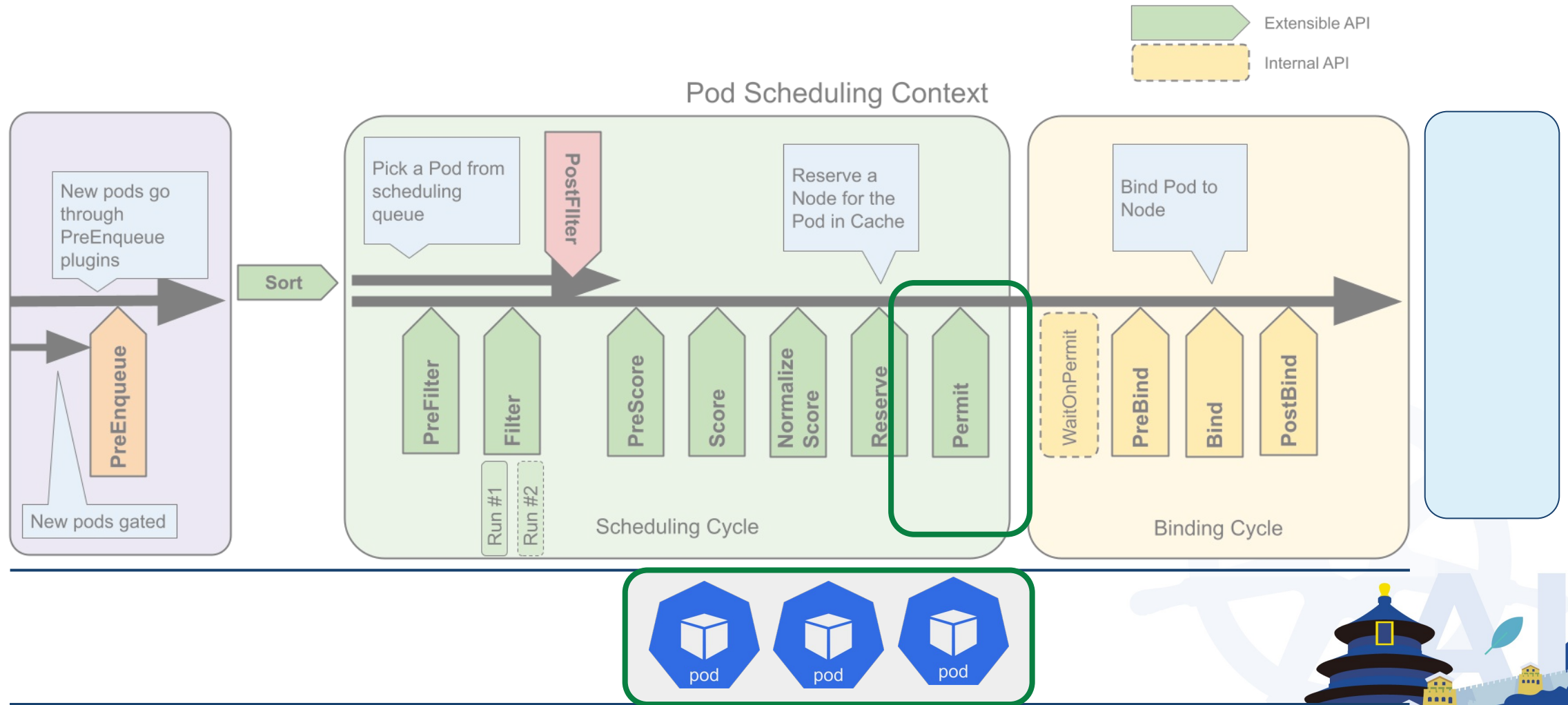
10-4



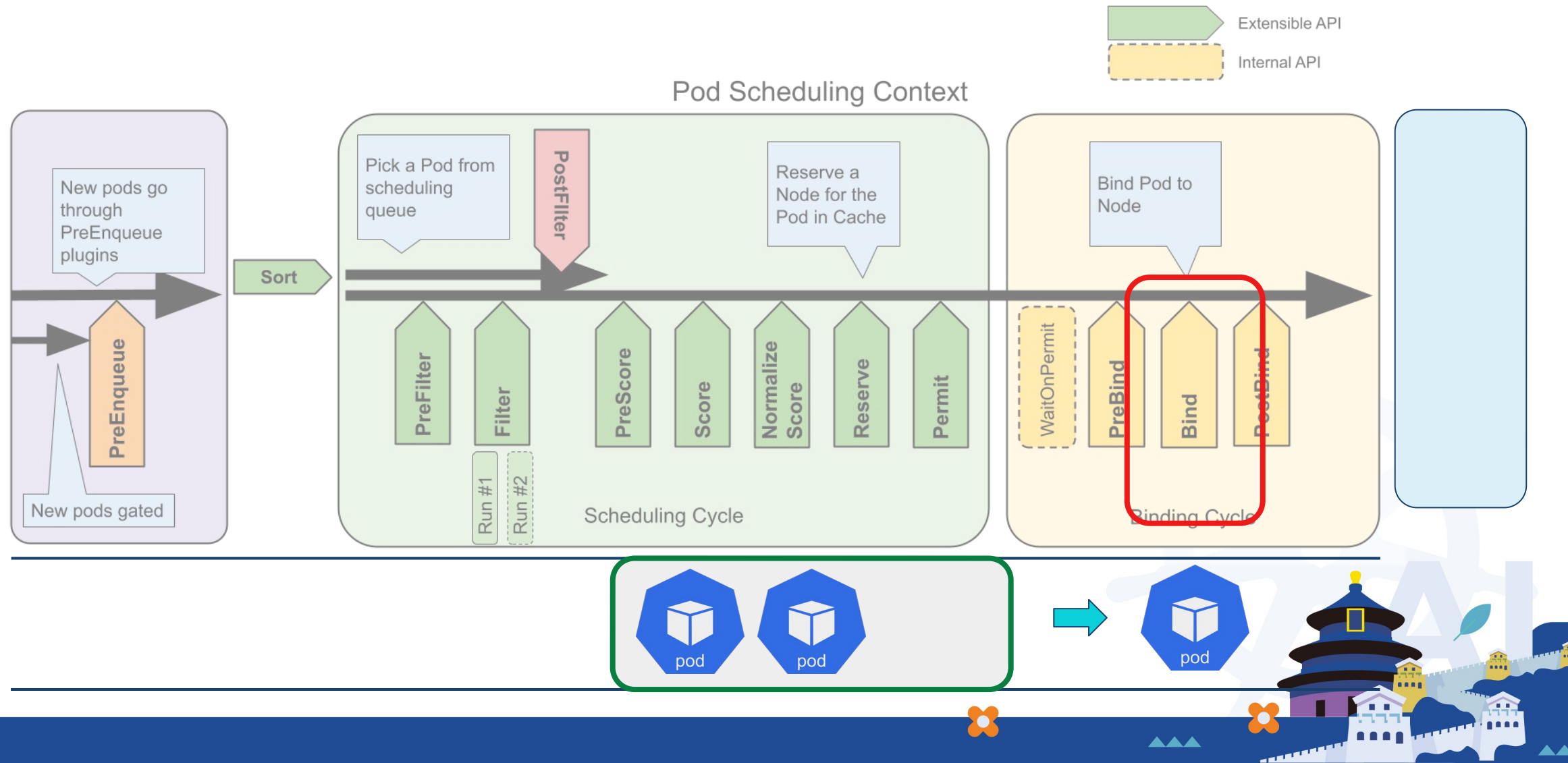
10-6

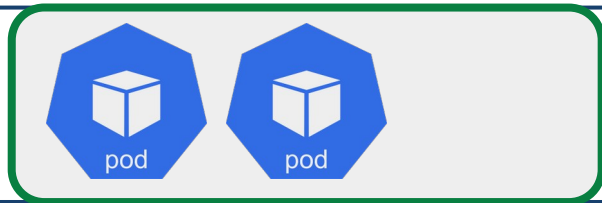
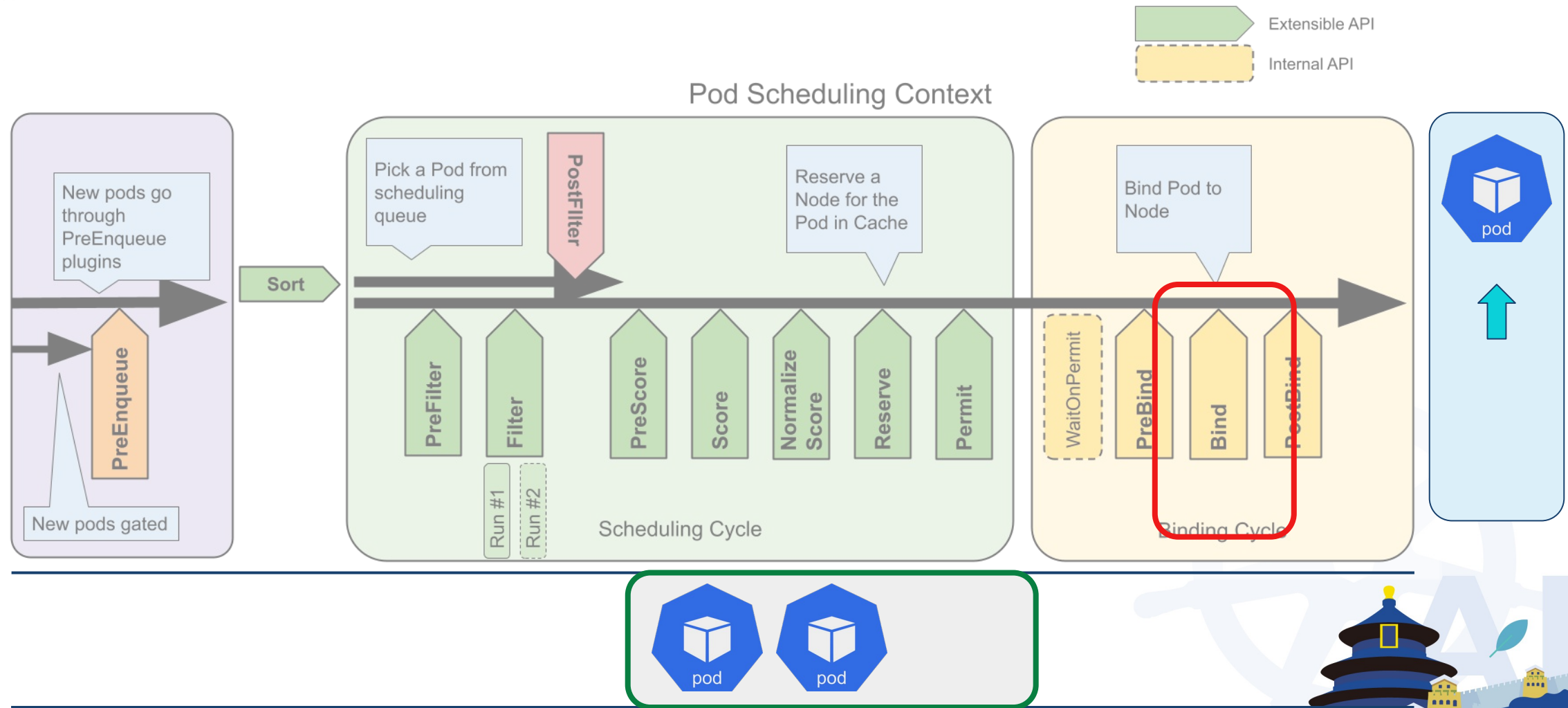


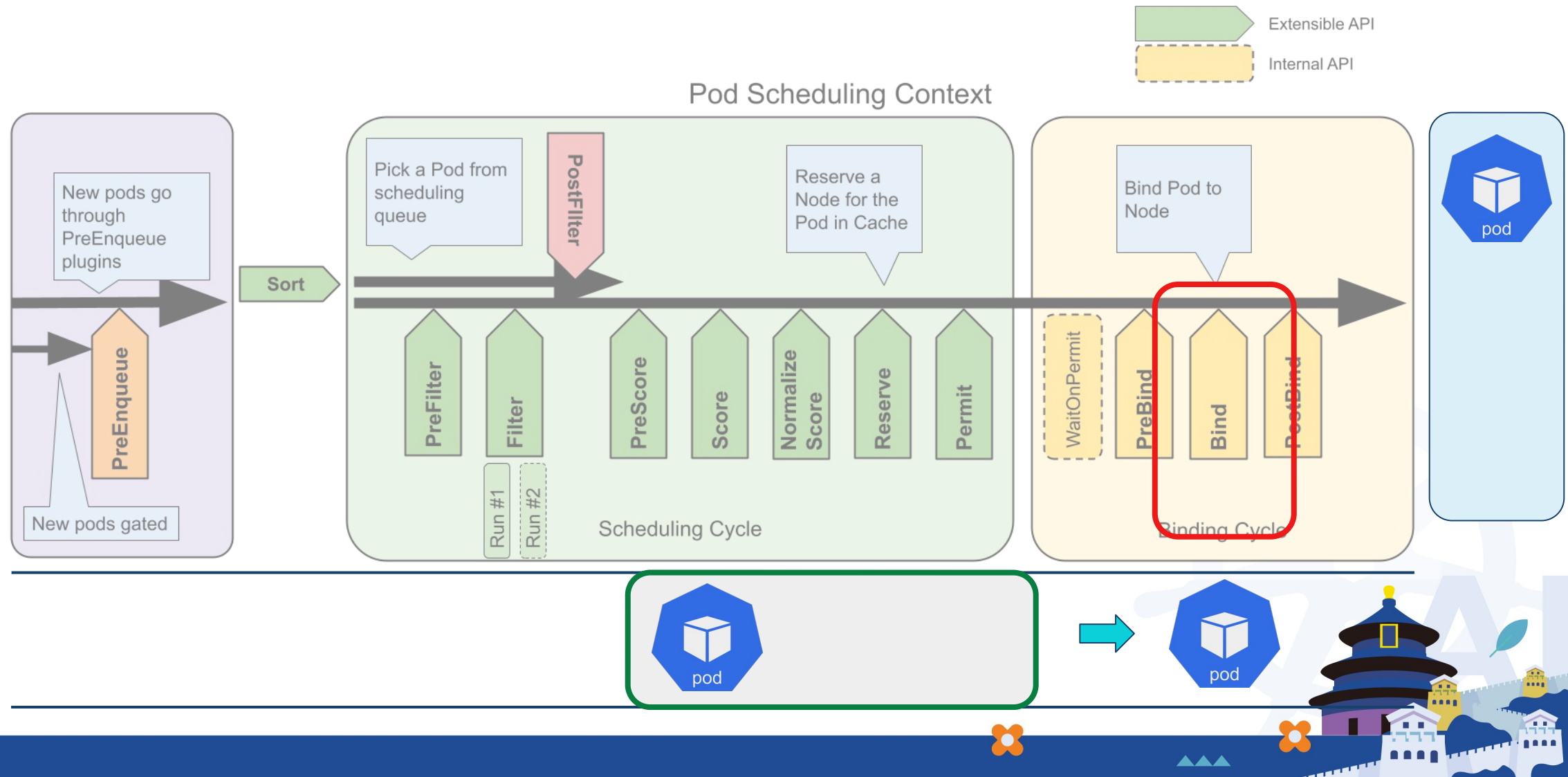
10-8



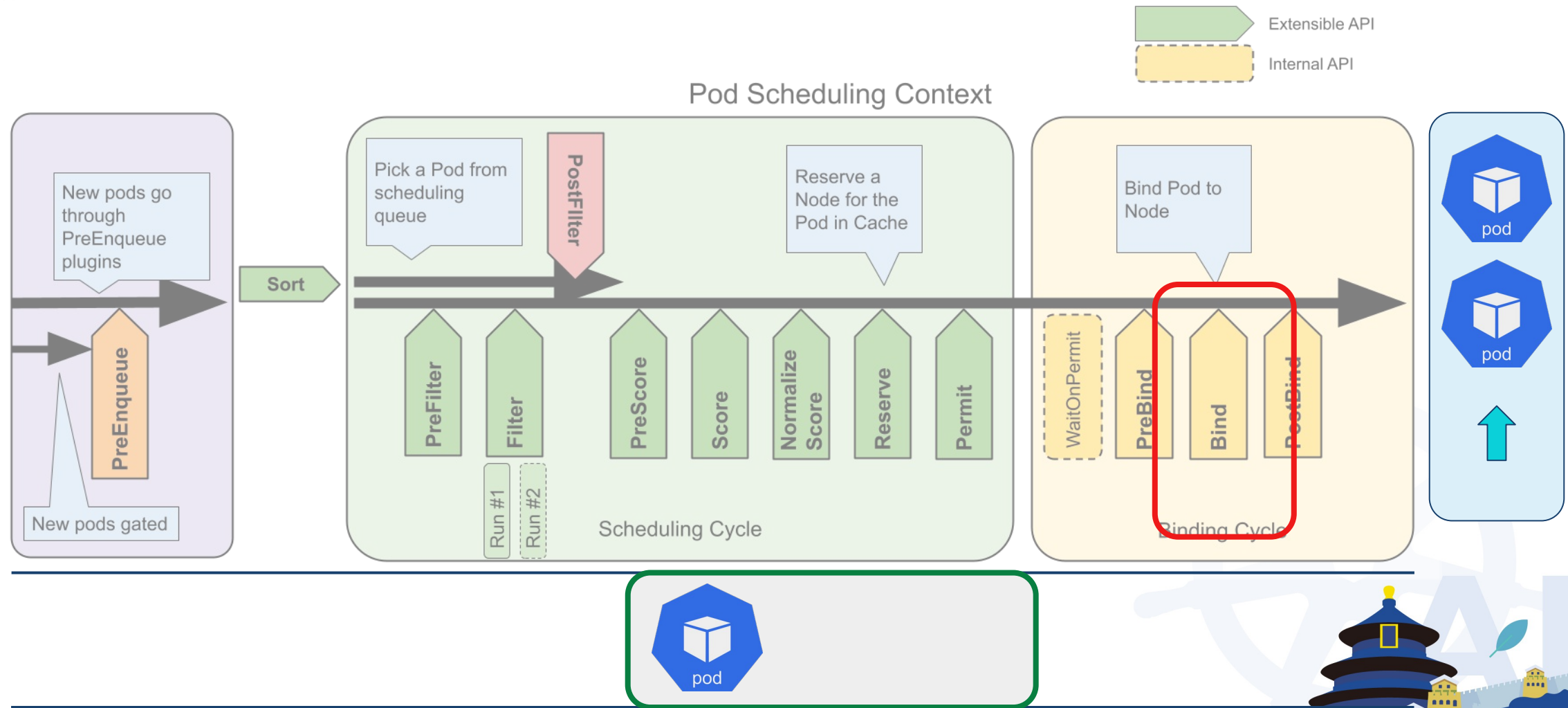
10-8

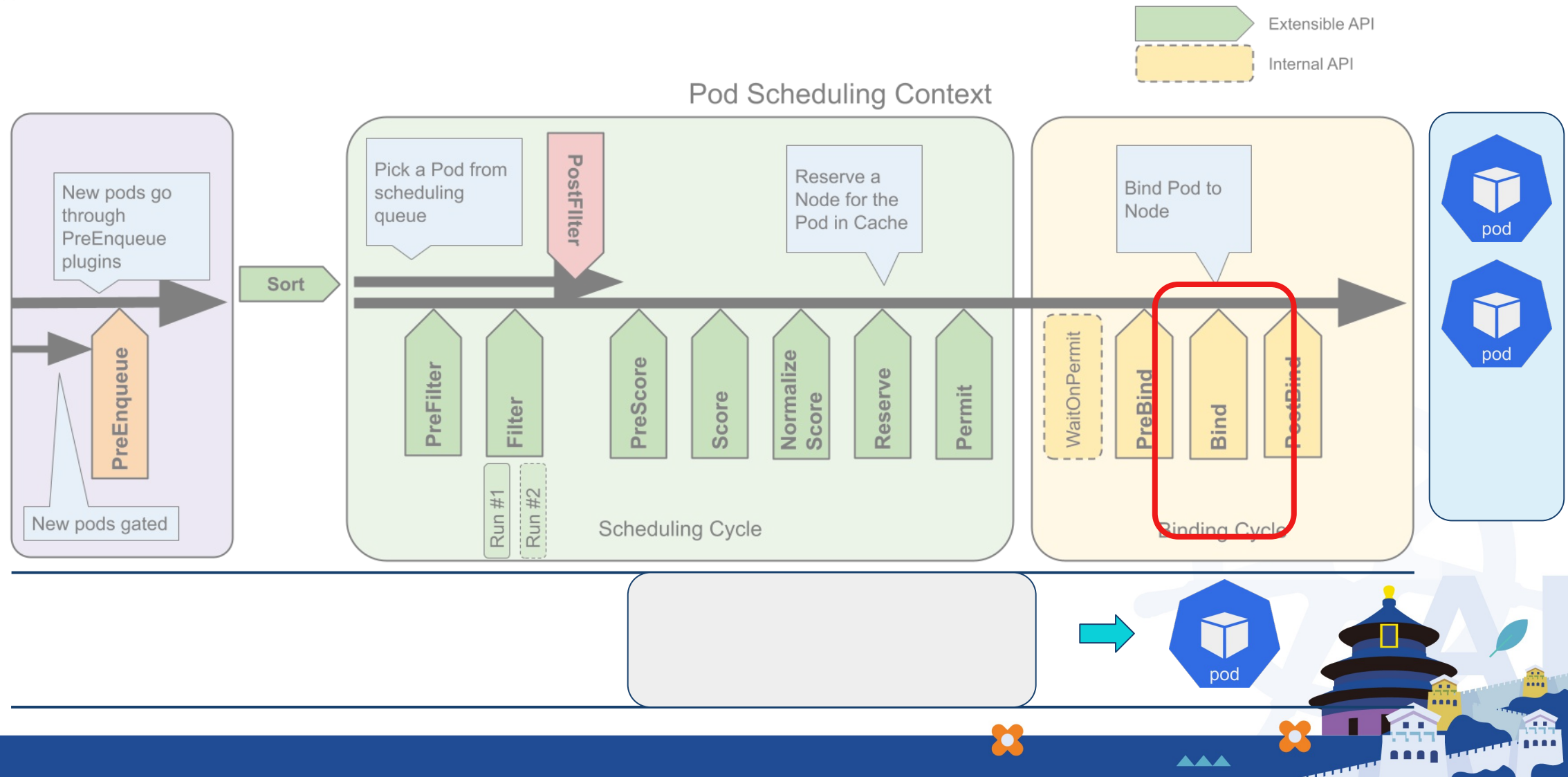




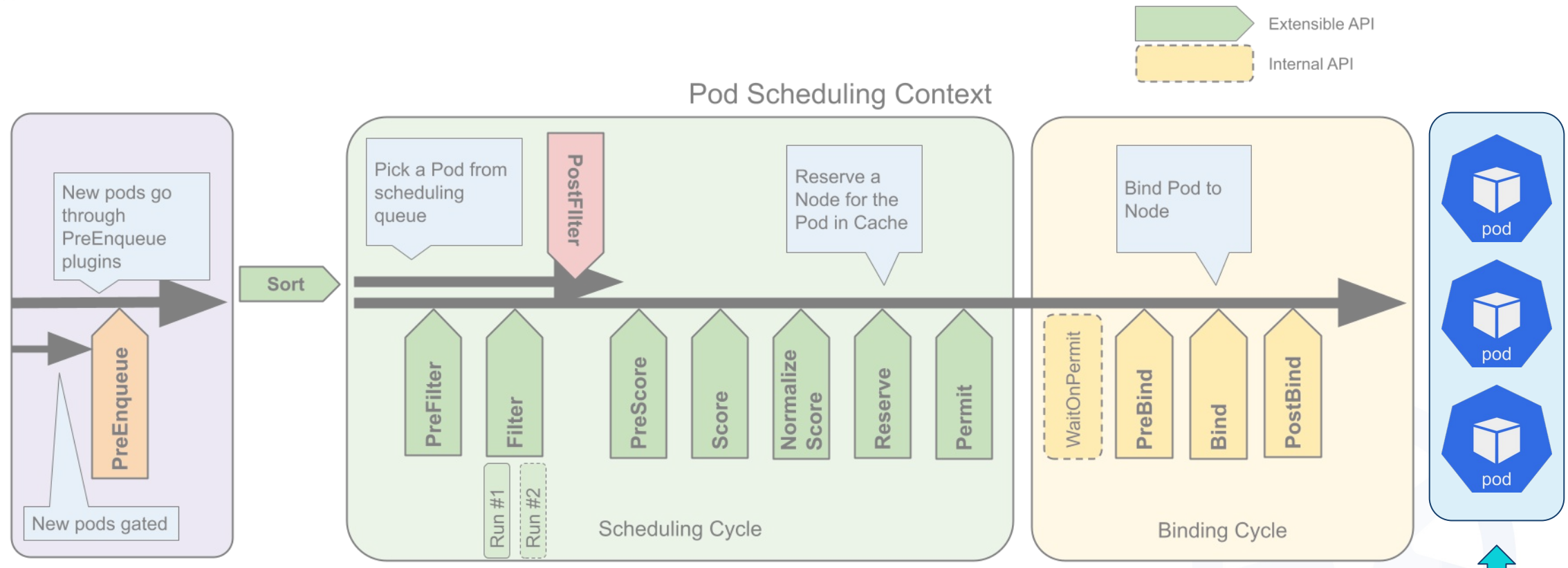


10-6





10-8



1.36



KEP-5832: Workload represents long-lived configuration-intent, whereas PodGroups represent transient units of scheduling.

Static Template

```
kind: Workload
spec:
  podGroupTemplate:
  - name: workers
    minCount: 8
```

Standalone Runtime Object

```
kind: PodGroup
metadata:
  name: workers-xyz
spec:
  minCount: 8
```

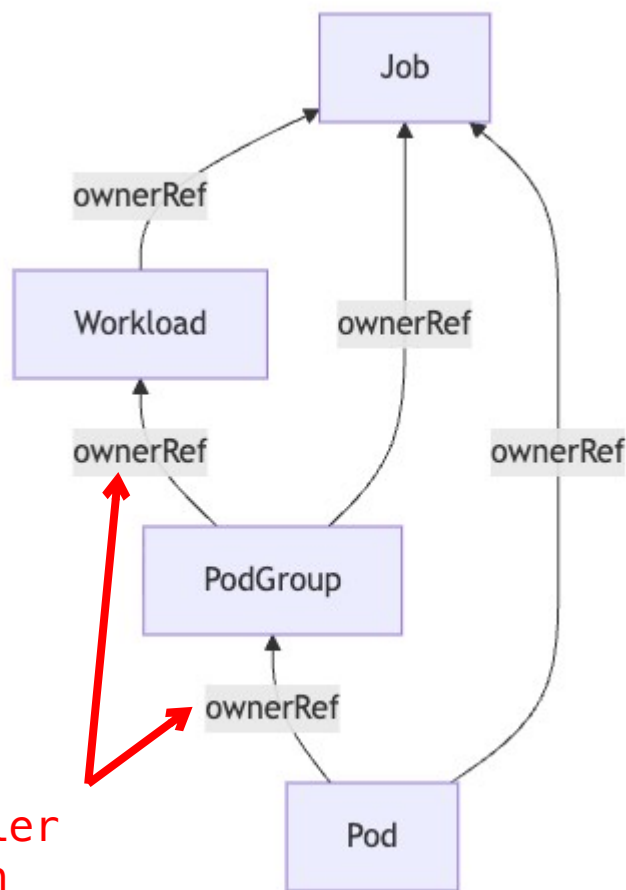
A Member of SchedGroup

```
kind: Pod
metadata:
  name: worker-0
spec:
  schedulingGroup: workers-xyz
```

controller handle workload and pod group.

Reference



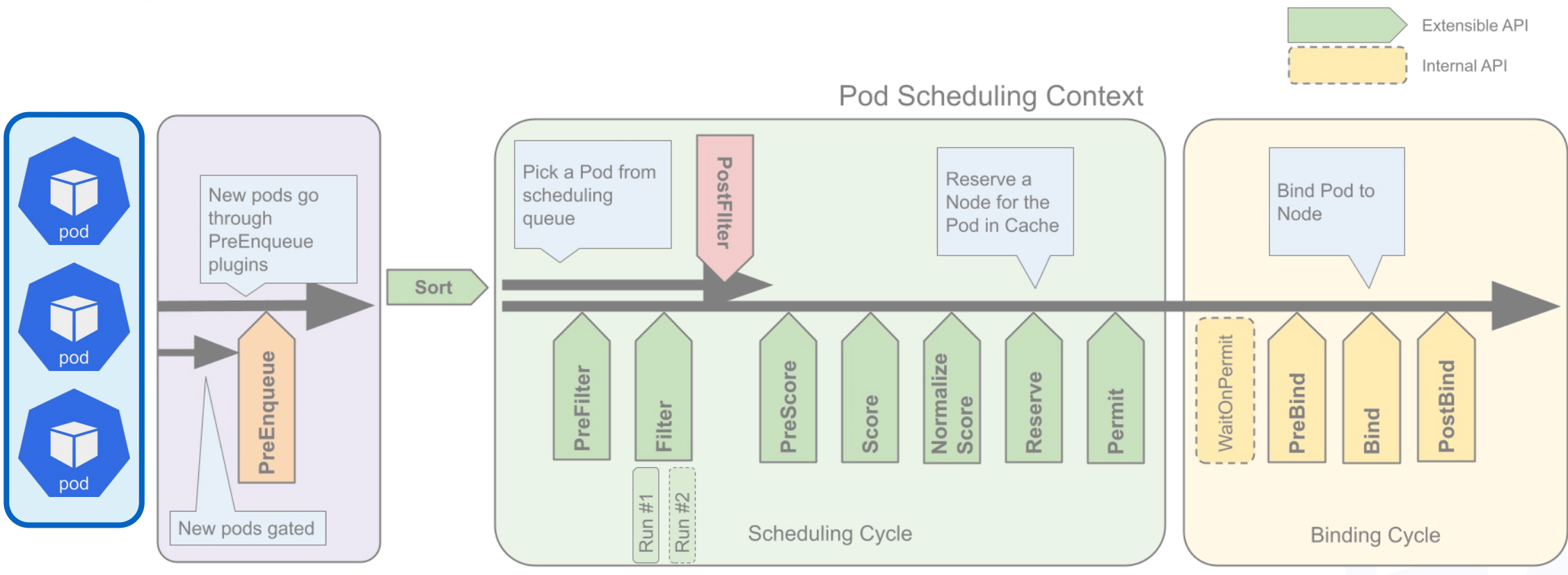


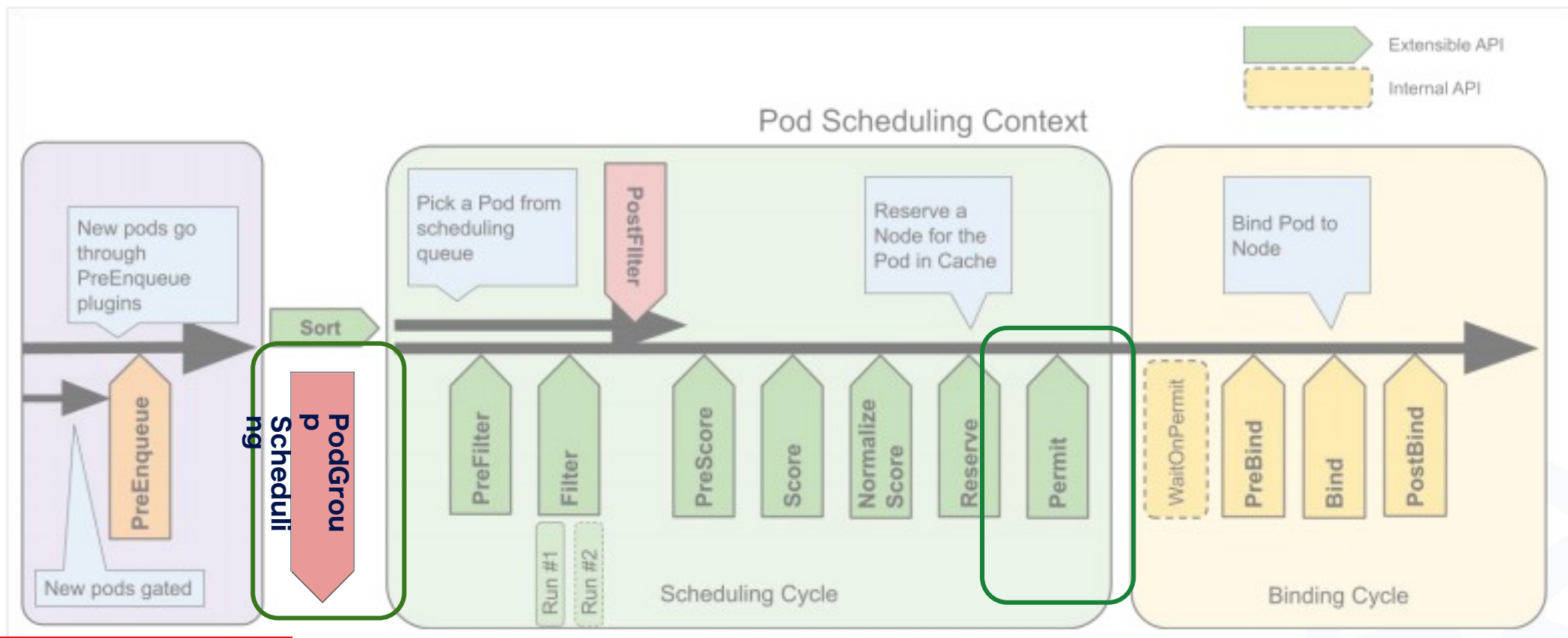
The controller field within ownerRef is false

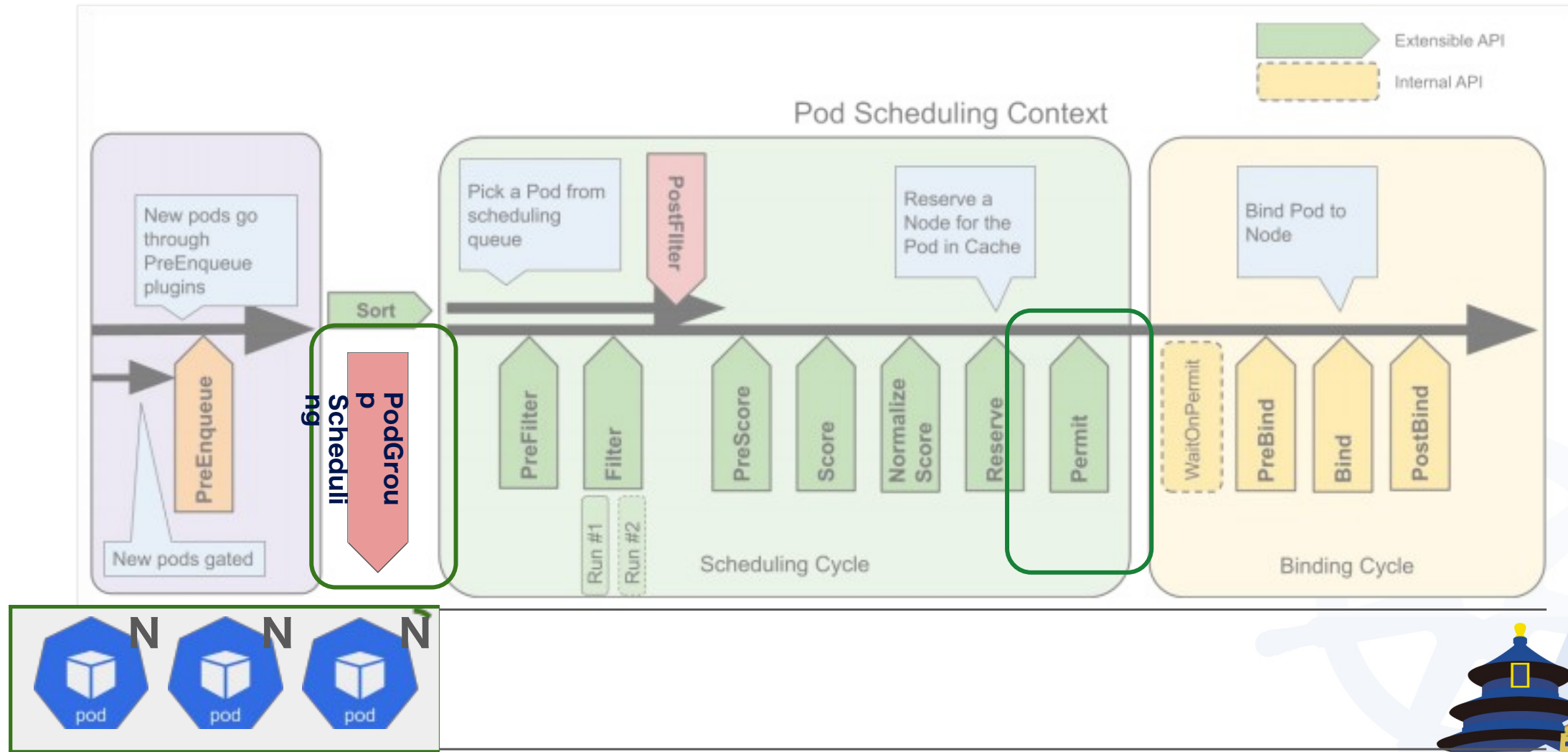
Gang Scheduling Constraints

```
kind Job
spec:
  parallelism: 8
  completions: 8
  completionMode: Indexed
```

- ✓ workload: `<job-name>--<hash>`
- ✓ pod group template name: `worker-0`
- ✓ pod group min count: `equals to Job.spec.parallelism`
- ✓ pod group name: `<workload-name>--<podGroup-template-name>--<hash>`
- ✓ pod group is protected via finalizer
- ✓ workload, pg, pods are controlled by job







KEP-5710:

- Priority: Higher priority units are always more important.
- Workload Type: Atomic workloads (PodGroups) are considered more important than individual Pods of the same priority.
- Start Time (for Single Pods): The one with the older StartTime is more important.
- Group Size (for PodGroups): The one with more members (larger size) is considered more important. If sizes are equal, the group that started earlier (has the oldest pod) is more important.

Static Template

```
kind: Workload
spec:
  podGroupTemplate:
    - name: workers
      priorityClassName: xxxx
      priority: 1
      disruptionMode: PodGroup
```

Standalone Runtime Object

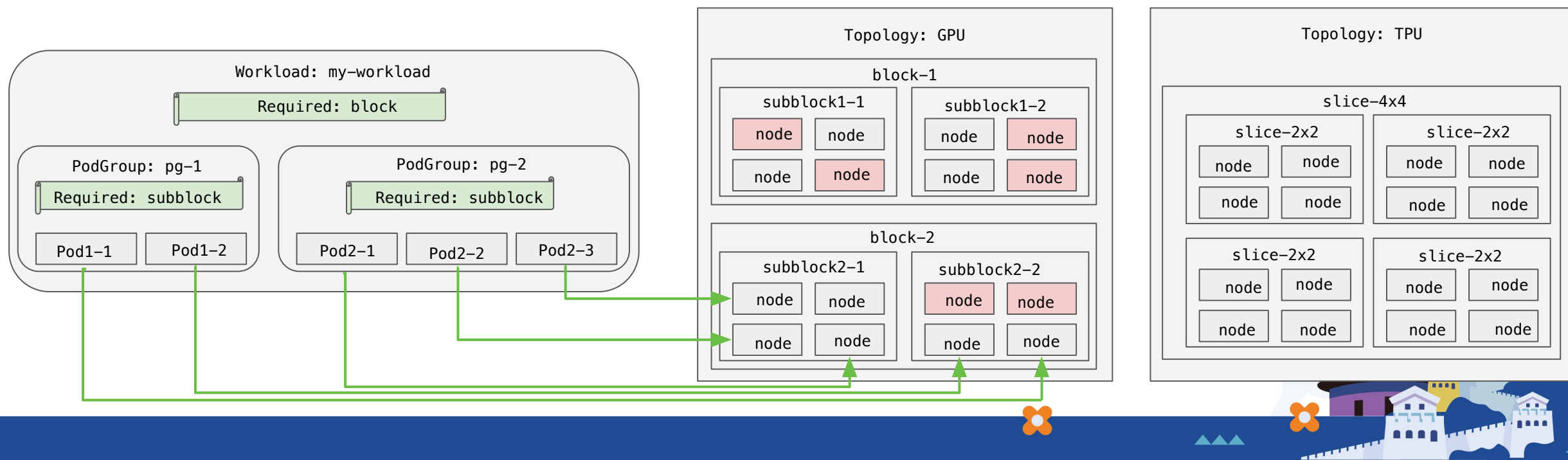
```
kind: PodGroup
metadata:
  name: workers-xyz
spec:
  priorityClassName: xxxx
  priority: 1
  disruptionMode: PodGroup
```



Topology-aware workload scheduling

KEP-5732:

- Workload forms a tree: Constraints (hard or soft) can be added all its tree nodes.
- Infrastructure forms a forest: Topology domains may have very different performance characteristics.
- Goal: Find the optimal placement of pod group into nodes.



Scaling DRA for infinite Workloads

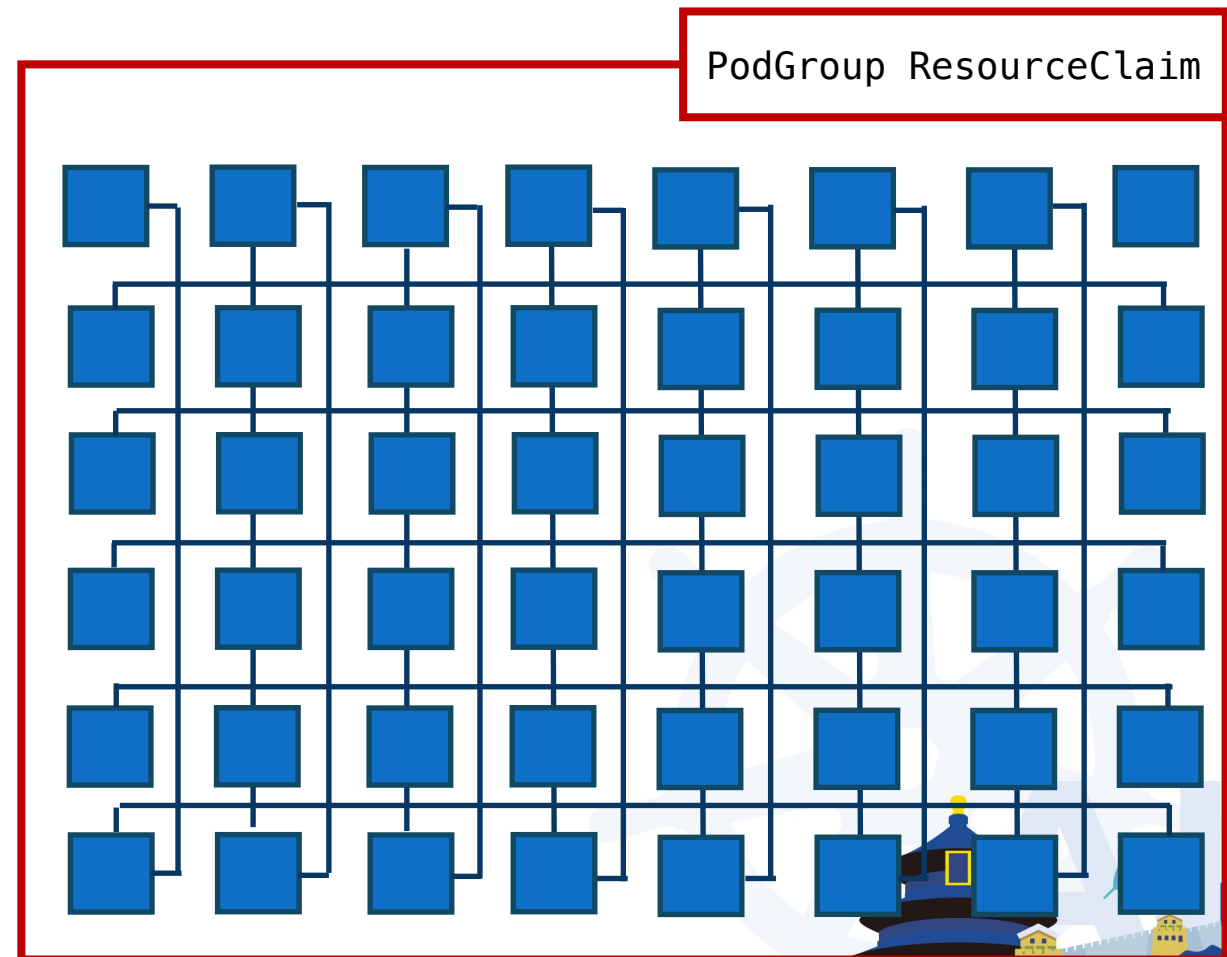
KEP-5729: Enabling infinite scaling and shared Dynamic Resource Allocation via PodGroups.

The Intergration: Merges the workload and PodGroup APIs with Dynamic Resource Allocation (DRA)

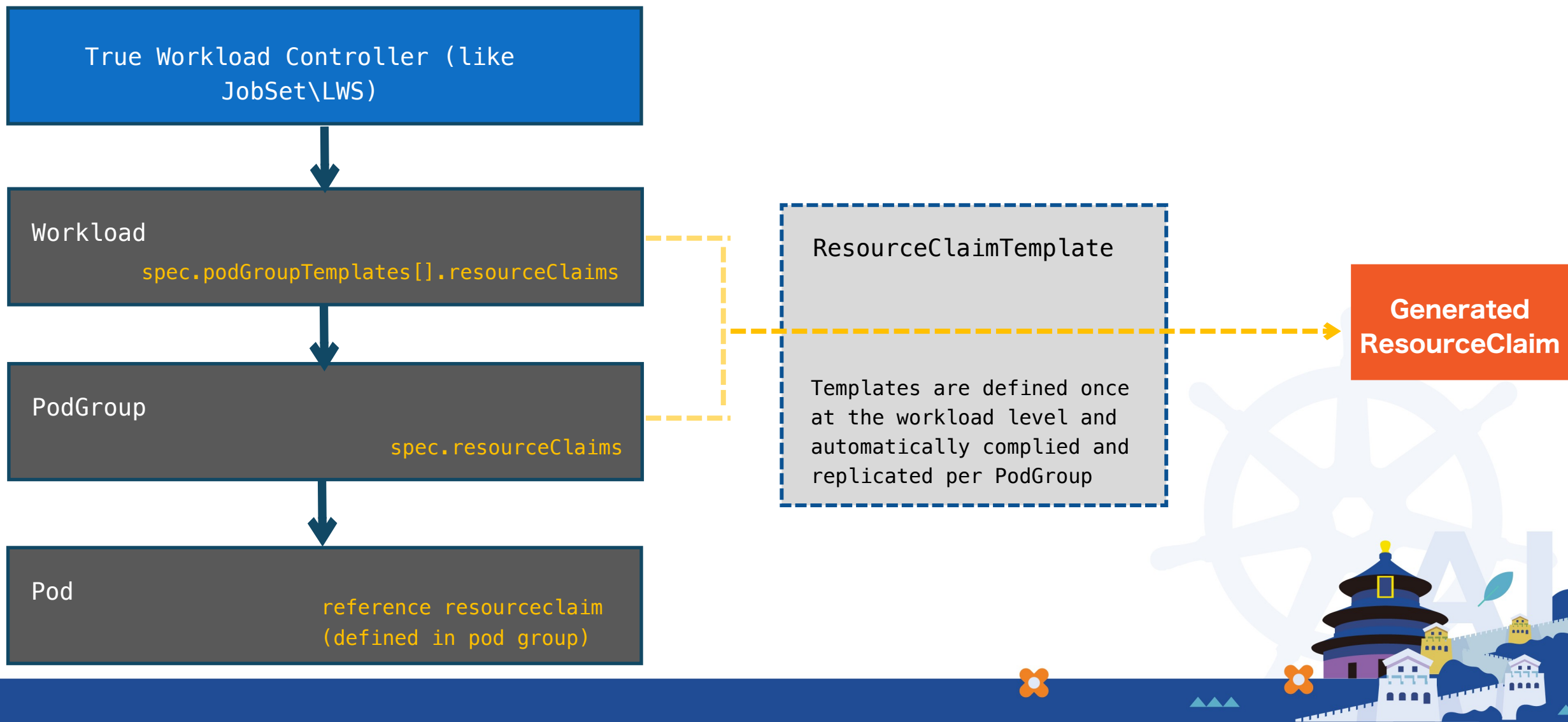
The Breakthrough: Eradicates the 256-pod array limit for shared resource claims, enabling infinite scaling.

The Automation: Centralizes claim lifecycle management for high-level APIs like JobSet and LeaderWorkerSet.

PodGroup ResourceClaim



Mapping the Workload API Extensions



1.37+



Next

KEP-5729: PDB for Multi-Pod Replicas.

<https://github.com/kubernetes/enhancements/pull/5671>

LWS RFC: Support kubernetes gang scheduling as a pod group provider

<https://github.com/kubernetes-sigs/lws/issues/666>

JobSet: Gang Scheduling of JobSets

<https://github.com/kubernetes-sigs/jobset/issues/969>

Doc: [Public] API Design for WAS Controller Integration:

https://docs.google.com/document/u/0/d/1VG7Zto9JYuPG4Anb01WMRryJlfV6met0jgob3T2NjZ4/mobilebasic?tab=t.0&_immersive_translate_auto_translate=1#heading=h.3s8c0yl3azl9



Thanks.

